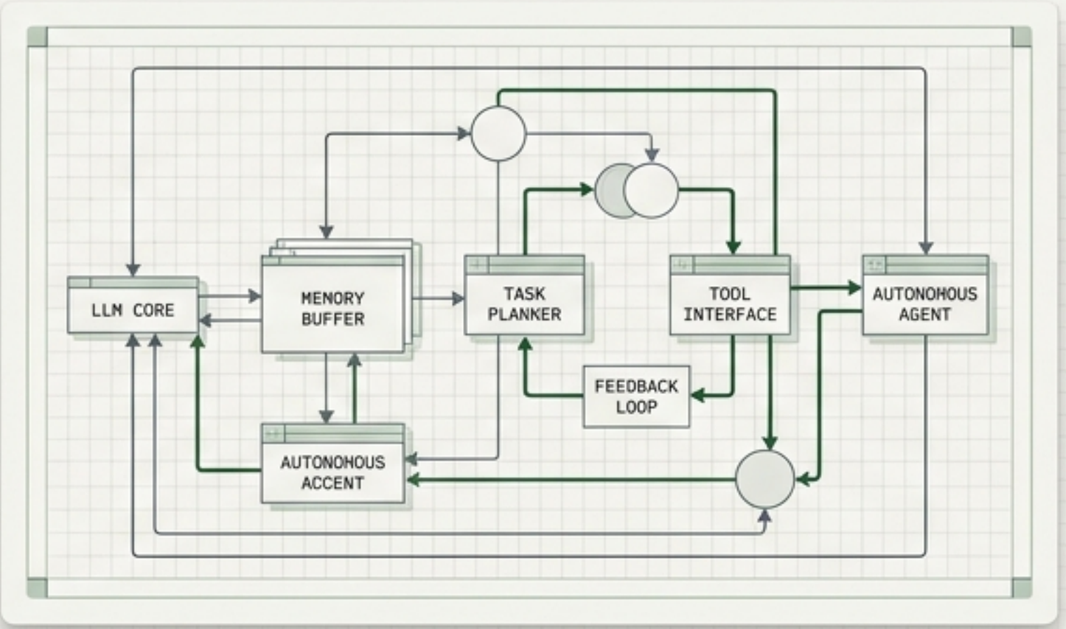
Agentic AI

The Evolution of Autonomy

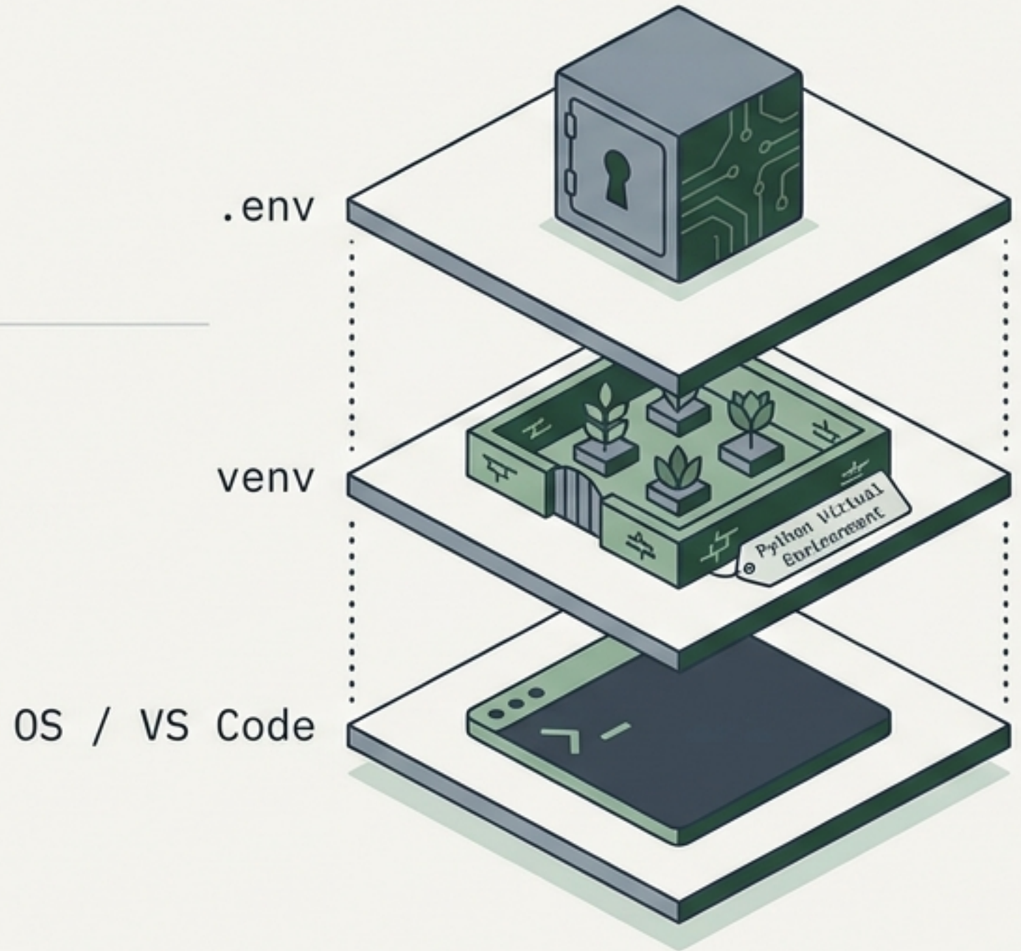
A technical explainer bridging the gap from deterministic LLM wrappers to autonomous multi-agent systems.



Week 1 Foundation & Architecture

The Foundation Stack

Production-grade agentic systems require rigorous dependency isolation and secret management. API keys must never be hardcoded; they are injected securely into the runtime environment via `python-dotenv`.



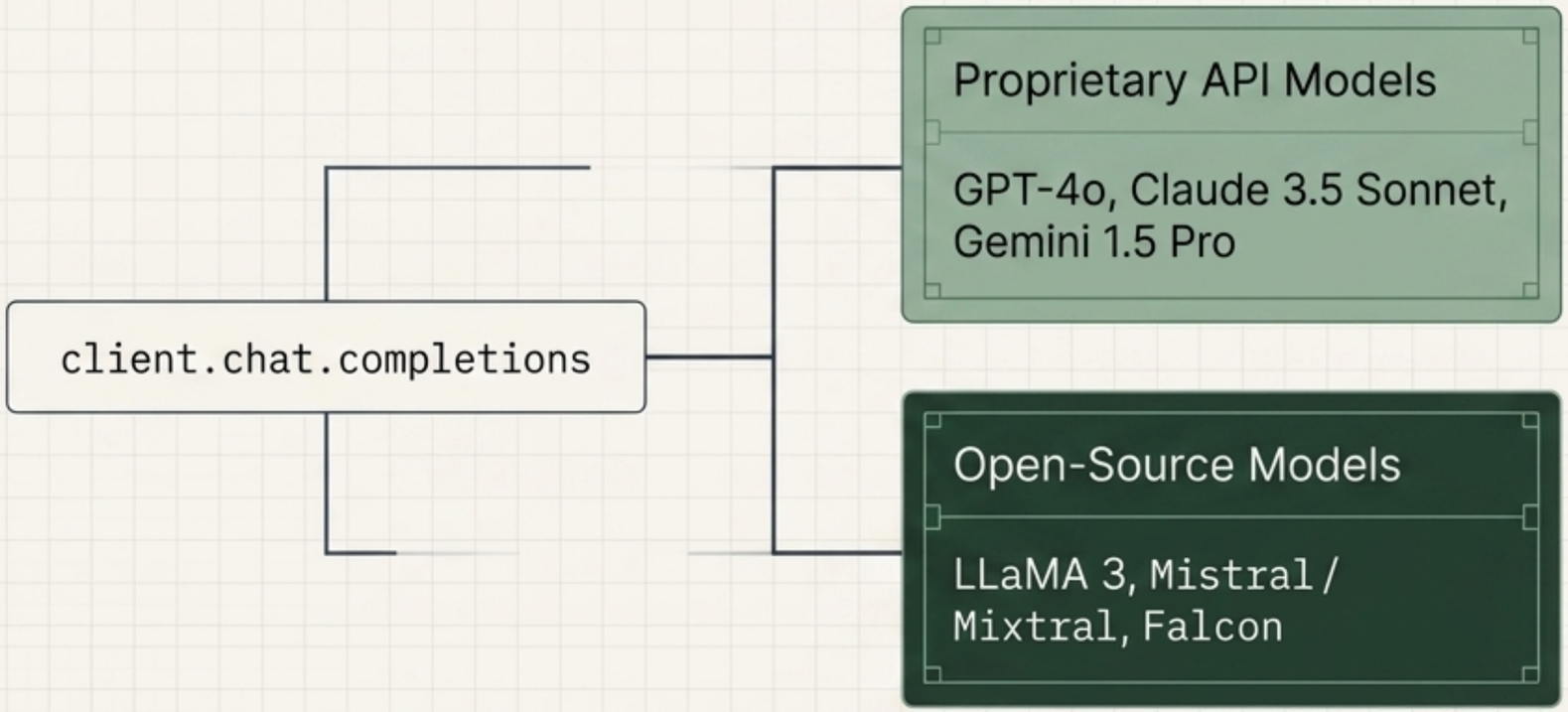
config.py

```
load_dotenv(override=True)

api_key = os.getenv('OPENAI_API_KEY')
```

Universal Provider Integration

The OpenAI client library establishes a de facto standard. By standardising the message format, developers can route tasks to different underlying models based on cost, capability, or latency without altering application logic.



Swap models with zero logic changes:

```
api_key = '...'
base_url = 'https://api.new-provider.com/v1'
```

Defining Agentic Architecture

An 'agent' is not a single model. It is a coordinated arrangement of models, tools, and execution environments where the LLM dynamically dictates the sequence of subsequent tasks.

The Autonomy Spectrum: Workflows vs. Agents

A system's categorisation depends entirely on who defines the execution path.

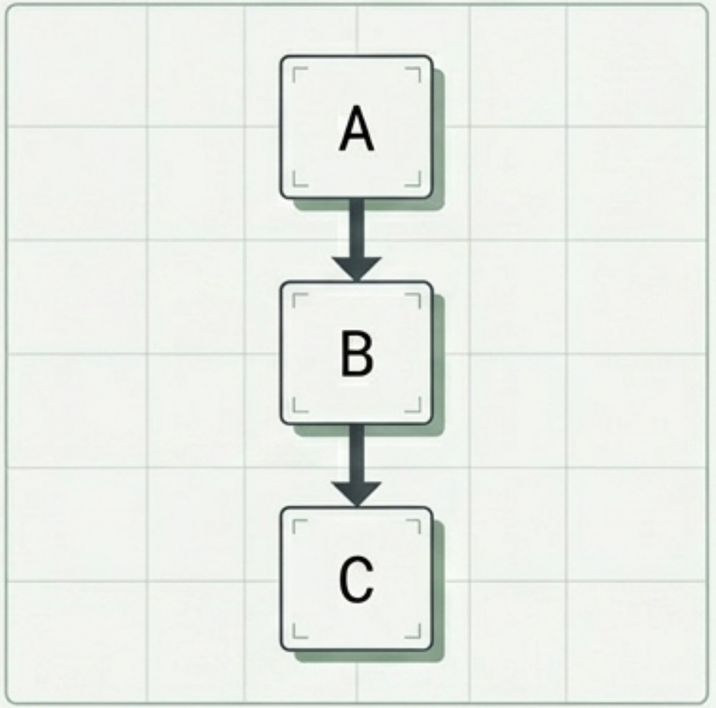
High Determinism → **High Autonomy**

	Workflows	Agents
Execution Path	Predefined, `deterministic`	Dynamic, `emergent at runtime`
Predictability	High, fully auditable	Lower, `non-deterministic` paths
Flexibility	Handles known, structured scenarios	Handles novel, open-ended tasks
Control	Developer-directed	Model-directed

Deterministic Topology: Workflow Patterns

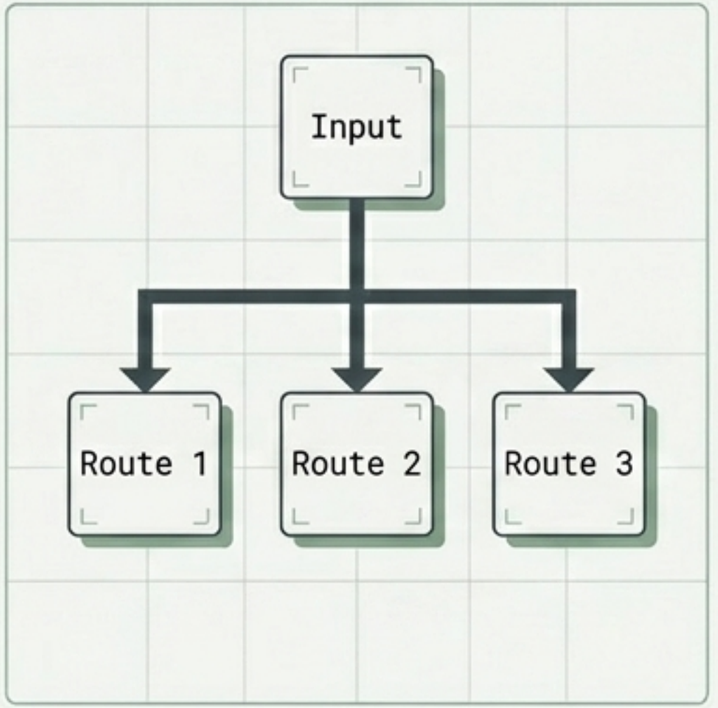
The foundational building blocks from which more sophisticated agentic architectures are constructed.

Chaining



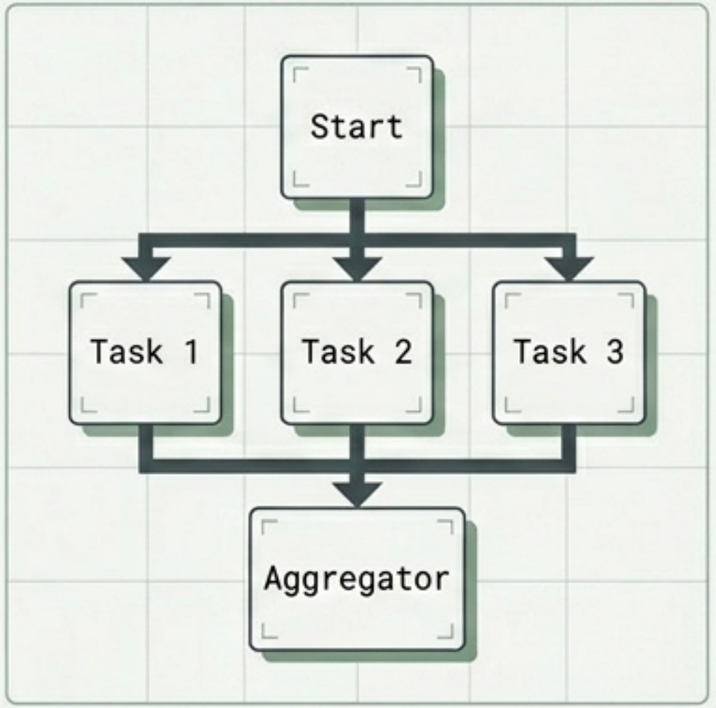
Output becomes input. Increases quality through focused steps.

Routing



Input categorised and directed to specialised sub-systems.

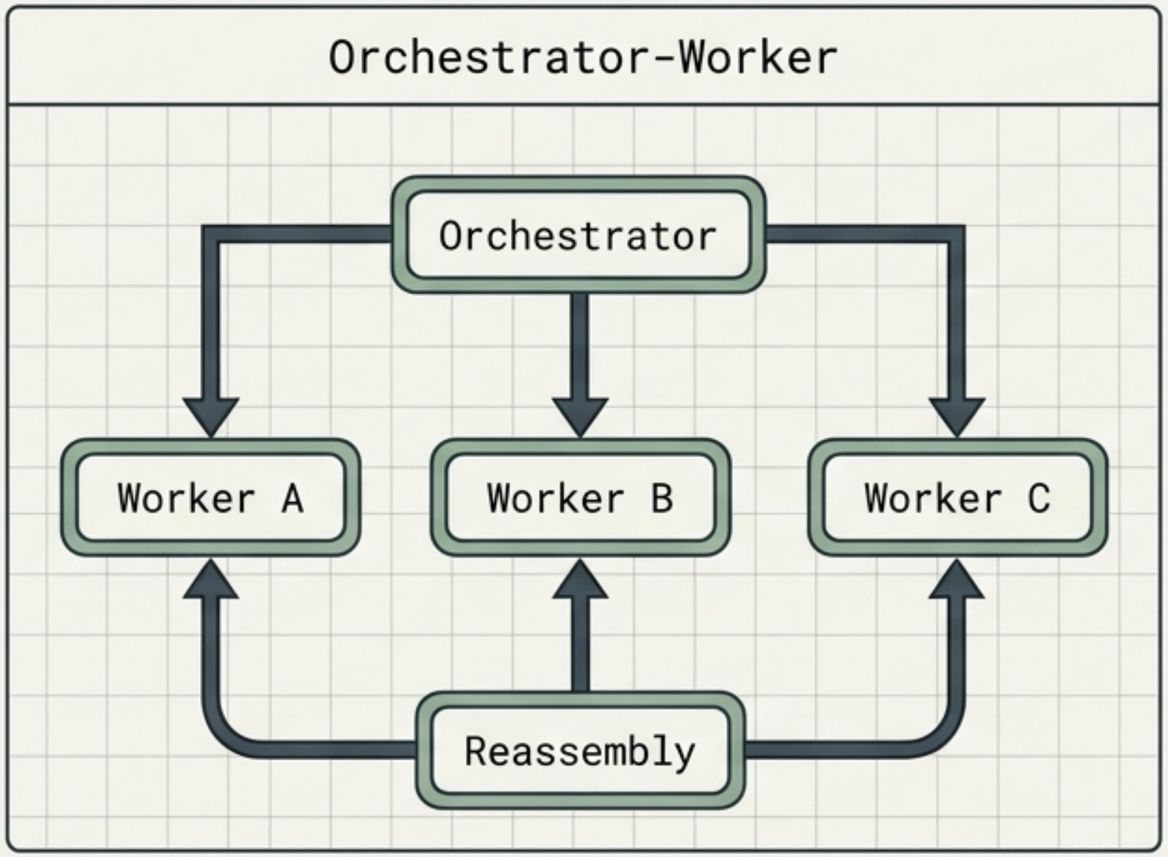
Parallelisation

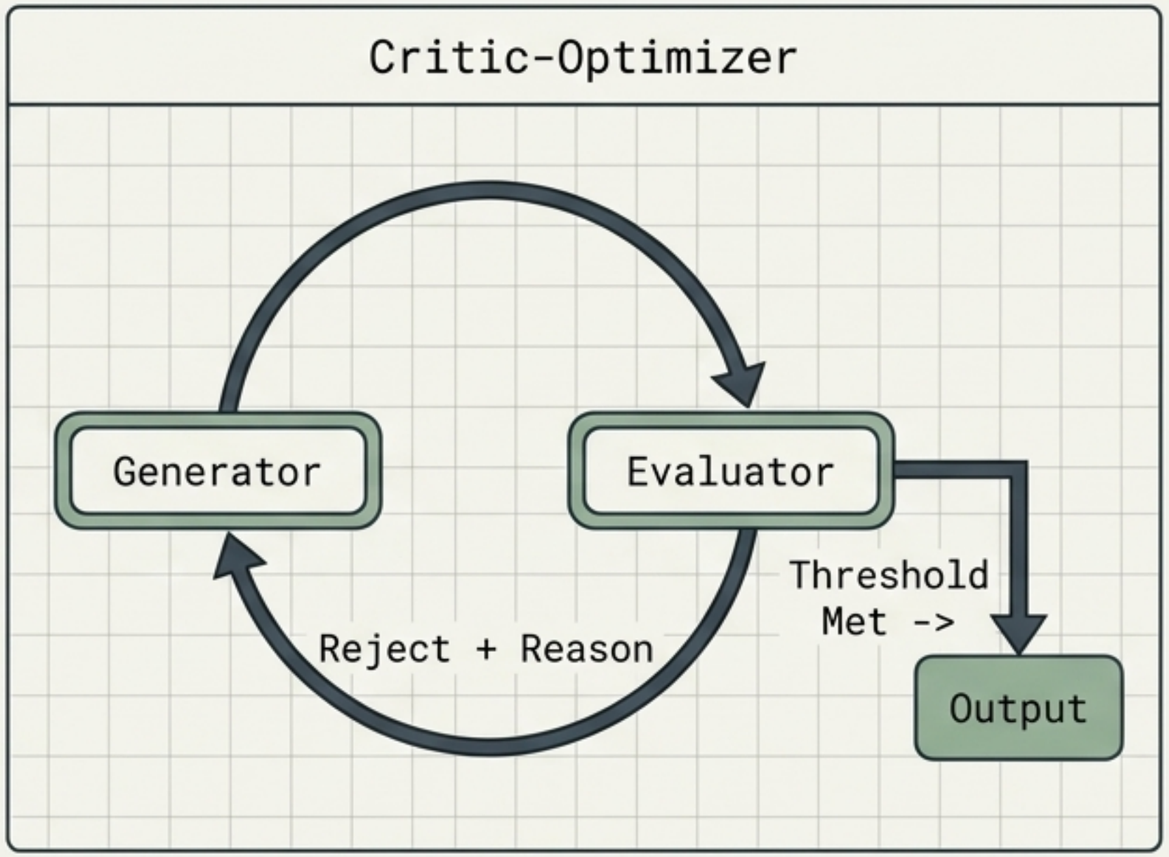


Independent execution for dramatically reduced latency.

Advanced Topology: Delegation & Alignment

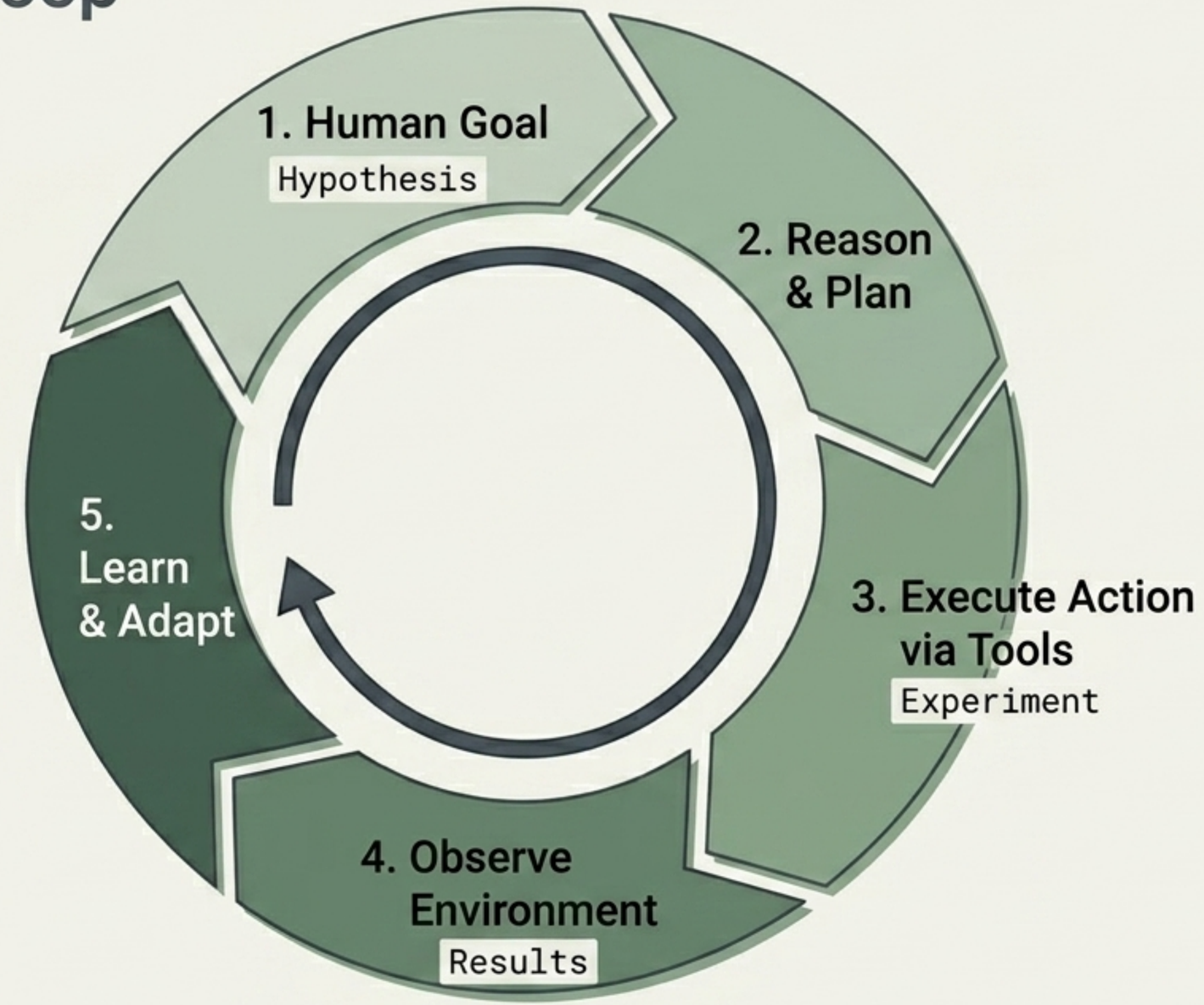
Why the Critic-Optimizer works: Alignment research proves LLMs are substantially better at **evaluating text** than generating perfect text on the first attempt. This pattern systematically leverages that asymmetry.





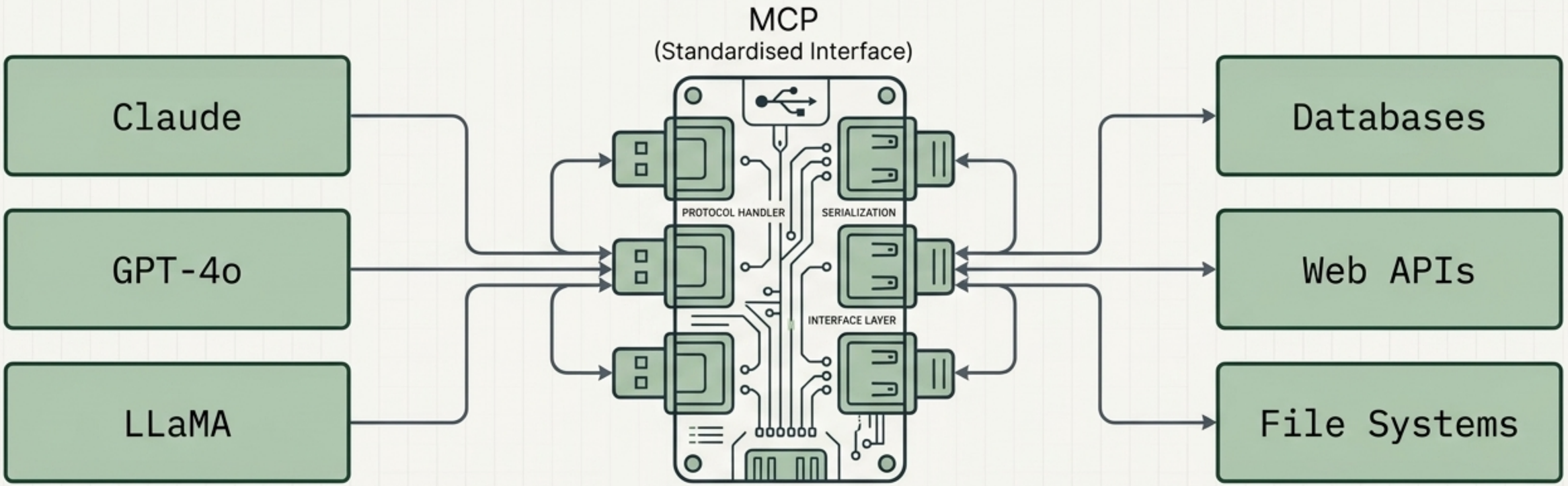
The Agentic Feedback Loop

Agentic systems abandon single-shot queries in favour of **iterative refinement**. They operate precisely like the **Scientific** for the **Scientific Method**: **hypothesise, experiment, observe, adapt, and repeat**.



Model Context Protocol (MCP)

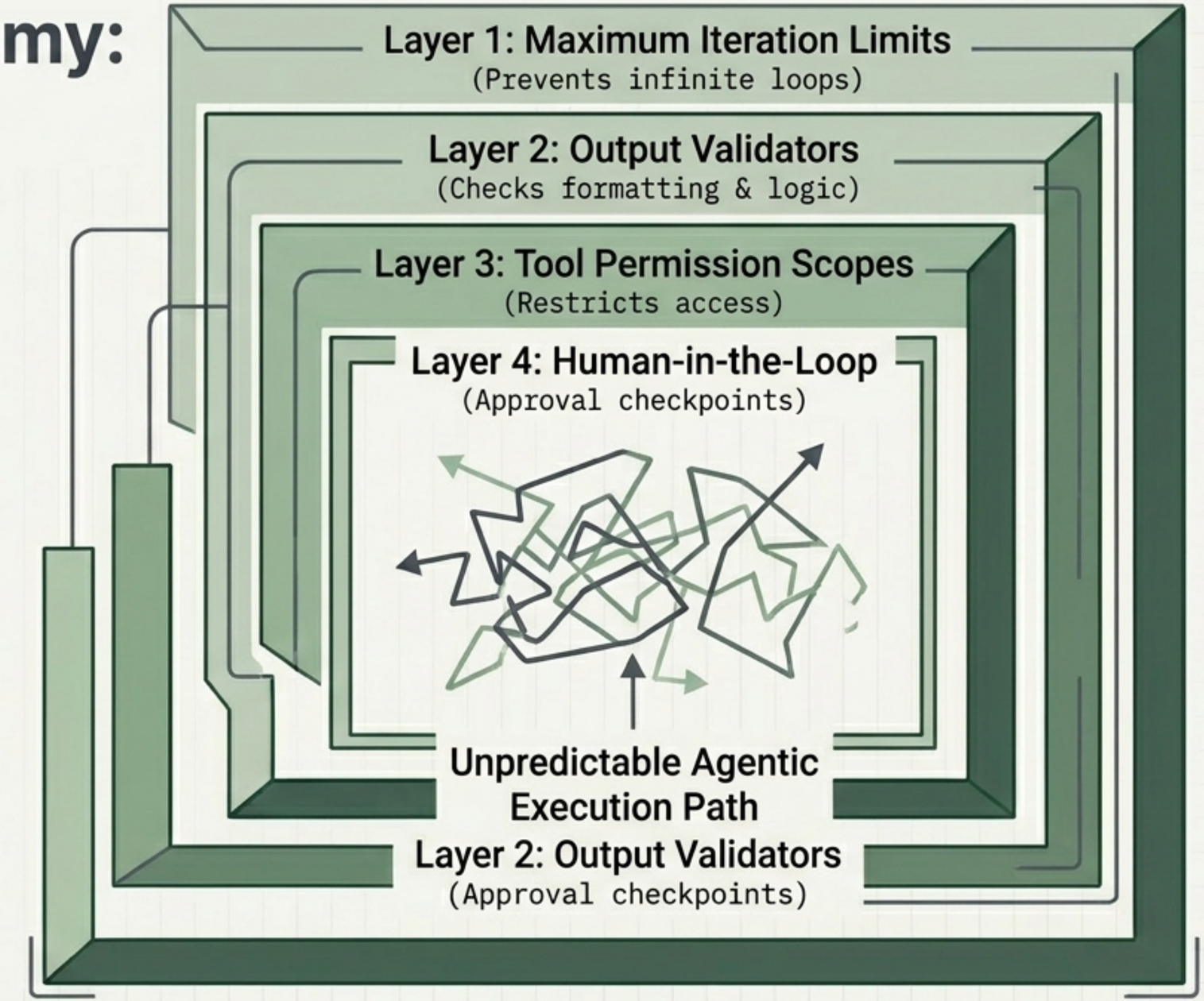
Anthropic’s open standard for LLM-to-tool connectivity.



Analogy: MCP is not an agent framework; it is a foundational protocol. Just as USB allows any peripheral to connect to any computer, MCP allows any compliant tool to connect to any compliant LLM, regardless of vendor.

Containing Autonomy: Risks & Guardrails

Emergent execution paths introduce unpredictability, compounding latency, and runaway API costs. Production-ready systems require **rigorous boundary layers** to ensure safe, auditable behaviour.



The Orchestration Landscape

AI frameworks abstract away the complexity of state management, tool integration, and agent coordination. Selection depends entirely on the required abstraction level and execution pattern.

CrewAI	**Type:** Multi-Agent Orchestration **Abstraction:** High **Profile:** Role-based team metaphor. Best for structured collaborative workflows.
LangGraph	**Type:** Orchestration Framework **Abstraction:** Low-Medium **Profile:** Directed graphs & shared state. Best for highly custom, non-linear pipelines.
AutoGen	**Type:** Multi-Agent Framework **Abstraction:** Medium **Profile:** Conversational interactions. Best for code generation & human-in-the-loop.

The Unified Agentic Architecture

From foundational environment variables to highly abstracted multi-agent crews, production-ready AI systems require seamless integration across every layer of the modern orchestration stack.